Build With Claude — Day 2

Building a Product with Claude Code: Skills, Subagents & Plugings

Author: Manaranjan Pradhan · www.manaranjanp.com

Lab: MCP server + Claude Agent SDK, with and without Skills

Start with a bare Claude API call to see what the model does *without* data access, build a Yahoo Finance MCP server, connect a Claude agent to it over HTTP, then build a second agent that adds Skills and watch the same queries produce consistently structured answers.

```
yfinance-agent/
├── ask_claude.py               # Step 0 — plain API call, no tools
├── yfinance_server.py          # Step 1 — FastMCP server (4 tools)
├── test_client.py              # Step 2 — client smoke test
├── agent_plain.py              # Step 3 — agent with MCP only
├── agent_skills.py             # Step 5 — agent with MCP + skills
├── .claude/skills/
│   ├── stock-snapshot/SKILL.md
│   ├── price-performance/SKILL.md
│   ├── price-performance/scripts/metrics.py
│   └── news-brief/SKILL.md
├── logs/                       # one event log per agent run
└── pyproject.toml              # uv project file (uv sync installs everything)
```

Prerequisites

Assumes `uv` and Claude Code (`claude`) are already installed, and Python 3.10+. Keep two terminals open: one for the MCP server, one for clients/agents.

macOS (zsh)

```
mkdir yfinance-agent && cd yfinance-agent
uv init --bare && uv add anthropic fastmcp yfinance claude-agent-sdk   # creates
pyproject.toml + .venv
# (or, with the lab zip unpacked: just `uv sync`)
export ANTHROPIC_API_KEY=sk-ant-...
```

Windows (PowerShell)

```
mkdir yfinance-agent; cd yfinance-agent
uv init --bare; uv add anthropic fastmcp yfinance claude-agent-sdk   # or `uv sync`
with the lab zip
$env:ANTHROPIC_API_KEY = "sk-ant-..."
```

The Agent SDK drives the installed Claude Code CLI; if you already use `claude` with your account you can skip the API key and it will use that login.

Opening a second terminal

`uv add` created `.venv/` inside the project folder. Any new terminal just needs to be in that folder — then either run scripts through `uv run` (no activation needed) or activate the venv once and use `python` directly. Both work; the guide uses `uv run`.

macOS (zsh)

```
cd /path/to/yfinance-agent
export ANTHROPIC_API_KEY=sk-ant-...        # env vars are per terminal — set again unless persisted

# option A: no activation
uv run test_client.py

# option B: activate once, then plain python
source .venv/bin/activate
python test_client.py
deactivate                                 # when done
```

Windows (PowerShell)

```
cd C:\path\to\yfinance-agent
$env:ANTHROPIC_API_KEY = "sk-ant-..."

# option A: no activation
uv run test_client.py

# option B: activate once, then plain python
.venv\Scripts\Activate.ps1
python test_client.py
deactivate
```

If PowerShell refuses to run `Activate.ps1` ("running scripts is disabled"), run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

To stop re-typing env vars in every terminal: macOS — add the `export` lines to `~/.zshrc`; Windows — `setx ANTHROPIC_API_KEY "sk-ant-..."` (takes effect in new terminals). `PROJECT_DIR` and `YF_MCP_URL` (Step 5) can be persisted the same way. To persist the API key across sessions, add the `export` line to `~/.zshrc` (macOS) or run `setx ANTHROPIC_API_KEY "sk-ant-..."` once and open a new terminal (Windows).

Step 0 — Ask Claude directly, with no tools

Before building anything, see what the model does on its own when asked for a live number. This is the baseline every later step improves on.

`ask_claude.py`

```
"""Step 0: plain Claude API call. No tools, no web search, no MCP.

    uv run ask_claude.py "What is the current share price of Reliance Industries
(RELIANCE.NS)?"
"""

import os
import sys

from anthropic import Anthropic

MODEL = os.environ.get("CLAUDE_MODEL", "claude-sonnet-5-5")

client = Anthropic()   # reads ANTHROPIC_API_KEY from the environment

question = " ".join(sys.argv[1:]) or "What is the current share price of Reliance
Industries (RELIANCE.NS)?"

response = client.messages.create(
    model=MODEL,
    max_tokens=600,
    system=(
        "You are a market-data analyst. Answer questions about listed companies "
        "with the latest price, market cap and P/E ratio."
    ),
    messages=[{"role": "user", "content": question}],
)

print(response.content[0].text)
print("-" * 60)
print(f"model={response.model}  stop={response.stop_reason}  "
      f"tokens: in={response.usage.input_tokens} out=
{response.usage.output_tokens}")
```

macOS

```
uv run ask_claude.py "What is the current share price of Reliance Industries
(RELIANCE.NS)?"
uv run ask_claude.py "What is Apple's market cap and P/E ratio right now?"
```

PowerShell

```
uv run ask_claude.py "What is the current share price of Reliance Industries
(RELIANCE.NS)?"
uv run ask_claude.py "What is Apple's market cap and P/E ratio right now?"
```

Typical output (yours will vary — run it twice):

```
I don't have access to real-time market data, so I can't give you Reliance's current share
price. As of my training data, RELIANCE.NS traded in the range of roughly INR 2,400–3,000,
but that may be well out of date. For the live price, check NSE, Yahoo Finance, or your broker.
------------------------------------------------------------
model=claude-sonnet-5-5  stop=end_turn  tokens: in=58 out=97
```

Sometimes it hedges like this; sometimes it states a specific figure with a date caveat; occasionally it states one without. None of these is a current price.

What to notice

Element	Meaning
`Anthropic()`	The API client. Reads `ANTHROPIC_API_KEY` from the environment.
`model`	Which Claude model answers. The lab reads it from `CLAUDE_MODEL`, default `claude-sonnet-5-5`.
`system`	Instructions that frame the whole conversation. Here it *tells* the model to report prices — and the model still cannot, because instructions don't grant data access.
`messages`	The conversation so far: one user turn.
`max_tokens`	Cap on the length of the reply.
`response.usage`	Input/output token counts — the same numbers you will see grow once tools are added.

Three things the output shows:

1. **The model has no live data.** Its knowledge stops at a training cutoff and it has no network. A system prompt cannot change that.
2. **The failure mode is not always a refusal.** When it does produce a number, that number is a recollection, not a lookup — and it will differ between runs.
3. **Everything that follows is about closing this gap.** An MCP server gives the model a way to *fetch* the number (Steps 1–4); skills then control *how* it fetches and reports (Steps 5–6).

Step 1 — Create the yfinance MCP server

`yfinance_server.py`

```
"""Yahoo Finance MCP server built with FastMCP.

Run (stdio):   python yfinance_server.py --stdio
Run (HTTP):    python yfinance_server.py                        ->
http://127.0.0.1:8000/mcp
Run (HTTPS):   python yfinance_server.py --https cert.pem key.pem
               (self-signed: openssl req -x509 -newkey rsa:2048 -nodes -keyout
key.pem -out cert.pem -days 365 -subj "/CN=localhost")
"""

import json
import sys

import yfinance as yf
from fastmcp import FastMCP

mcp = FastMCP(
    name="yfinance",
    instructions="Tools for fetching stock data from Yahoo Finance by ticker symbol
(e.g. AAPL, RELIANCE.NS).",
)


@mcp.tool
def get_historical_stock_prices(ticker: str, period: str = "1mo", interval: str =
"1d") -> str:
    """Get historical OHLCV data for a stock.

    Args:
        ticker: Ticker symbol, e.g. "AAPL" or "TCS.NS".
        period: 1d,5d,1mo,3mo,6mo,1y,2y,5y,10y,ytd,max (default 1mo).
        interval: 1m,2m,5m,15m,30m,60m,90m,1h,1d,5d,1wk,1mo,3mo (default 1d).
                  Intraday intervals are limited to the last 60 days.
    Returns JSON list of records: Date, Open, High, Low, Close, Volume, Dividends,
Stock Splits.
    """
    try:
        df = yf.Ticker(ticker).history(period=period, interval=interval)
        if df.empty:
            return f"No price data found for {ticker} (period={period}, interval=
{interval})."
        return df.reset_index(names="Date").to_json(orient="records",
date_format="iso")
    except Exception as e:
        return f"Error fetching historical prices for {ticker}: {e}"


@mcp.tool
def get_stock_info(ticker: str) -> str:
    """Get comprehensive stock data: current price, valuation metrics, margins,
    dividends, balance-sheet summary, analyst targets and company details.
```

```
    Args:
        ticker: Ticker symbol, e.g. "AAPL".
    Returns JSON object of all fields Yahoo Finance exposes for the ticker.
    """
    try:
        info = yf.Ticker(ticker).info
        if not info or "symbol" not in info:
            return f"No stock info found for {ticker}."
        return json.dumps(info, default=str)
    except Exception as e:
        return f"Error fetching stock info for {ticker}: {e}"


@mcp.tool
def get_yahoo_finance_news(ticker: str) -> str:
    """Get the latest news articles for a stock.

    Args:
        ticker: Ticker symbol, e.g. "AAPL".
    Returns JSON list of {title, summary, publisher, published, url}.
    """
    try:
        news = yf.Ticker(ticker).news
        items = []
        for n in news or []:
            c = n.get("content", {})
            if c.get("contentType") != "STORY":
                continue
            items.append(
                {
                    "title": c.get("title"),
                    "summary": c.get("summary"),
                    "publisher": c.get("provider", {}).get("displayName"),
                    "published": c.get("pubDate"),
                    "url": c.get("canonicalUrl", {}).get("url"),
                }
            )
        if not items:
            return f"No news found for {ticker}."
        return json.dumps(items)
    except Exception as e:
        return f"Error fetching news for {ticker}: {e}"


@mcp.tool
def get_stock_actions(ticker: str) -> str:
    """Get dividend and stock-split history for a stock.

    Args:
        ticker: Ticker symbol, e.g. "AAPL".
    Returns JSON list of records: Date, Dividends, Stock Splits.
    """
```

```
    try:
        df = yf.Ticker(ticker).actions
        if df is None or df.empty:
            return f"No dividend/split history found for {ticker}."
        return df.reset_index(names="Date").to_json(orient="records",
date_format="iso")
    except Exception as e:
        return f"Error fetching stock actions for {ticker}: {e}"


if __name__ == "__main__":
    args = sys.argv[1:]
    if "--stdio" in args:
        mcp.run()
    elif "--https" in args:
        i = args.index("--https")
        cert, key = args[i + 1], args[i + 2]
        mcp.run(transport="http", host="127.0.0.1", port=8000,
                uvicorn_config={"ssl_certfile": cert, "ssl_keyfile": key})
    else:
        mcp.run(transport="http", host="127.0.0.1", port=8000)
```

What to notice

Element	Meaning
`FastMCP(name, instructions)`	`name` is the server id clients see; `instructions` is a hint the client can show the model.
`@mcp.tool`	Registers the function as an MCP tool. Function name → tool name; type hints → JSON input schema; docstring → tool description the model reads. Write it for the model.
Return `str`	Tool results are text. We return JSON strings; on failure we return an error *string* instead of raising, so the model gets a readable message rather than a protocol error.
`mcp.run(transport="http", ...)`	Streamable-HTTP transport at `http://127.0.0.1:8000/mcp`. `--stdio` is the subprocess mode Claude Desktop uses. `--https cert.pem key.pem` passes TLS files to uvicorn (self-signed cert: `openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 365 -subj "/CN=localhost"`; on Windows use Git Bash or `winget install OpenSSL`).

Step 2 — Run the server and test it with a client

Terminal 1:

macOS

```
uv run yfinance_server.py
```

PowerShell

```
uv run yfinance_server.py
```

Look for `Starting MCP server 'yfinance' with transport 'http'`.

`test_client.py`

```python
"""Smoke-test the yfinance MCP server over HTTP.   python test_client.py"""

import asyncio
import json
import os

from fastmcp import Client

MCP_URL = os.environ.get("YF_MCP_URL", "http://127.0.0.1:8000/mcp")
TICKER = "AAPL"


def parse(result):
    raw = result.content[0].text
    try:
        return json.loads(raw)
    except json.JSONDecodeError:
        print(f"  server said: {raw}")
        return None


async def main():
    async with Client(MCP_URL) as client:
        tools = await client.list_tools()
        print("Tools:", [t.name for t in tools], "\n")

        r = await client.call_tool("get_historical_stock_prices", {"ticker":
TICKER, "period": "5d"})
        if rows := parse(r):
            print(f"get_historical_stock_prices -> {len(rows)} rows, last close=
{rows[-1]['Close']:.2f}")

        r = await client.call_tool("get_stock_info", {"ticker": TICKER})
        if info := parse(r):
            print(f"get_stock_info -> {info.get('longName')} | price=
{info.get('currentPrice')}")

        r = await client.call_tool("get_yahoo_finance_news", {"ticker": TICKER})
        if news := parse(r):
            print(f"get_yahoo_finance_news -> {len(news)} articles, first: {news[0]
['title']}")

        r = await client.call_tool("get_stock_actions", {"ticker": TICKER})
        if actions := parse(r):
            print(f"get_stock_actions -> {len(actions)} events, latest:
{actions[-1]}")


asyncio.run(main())
```

Terminal 2:

macOS

```
uv run test_client.py
```

PowerShell

```
uv run test_client.py
```

Expected:

```
Tools: ['get_historical_stock_prices', 'get_stock_info', 'get_yahoo_finance_news',
'get_stock_actions']

get_historical_stock_prices -> 5 rows, last close=232.40
get_stock_info -> Apple Inc. | price=232.4
get_yahoo_finance_news -> 8 articles, first: Apple reports ...
get_stock_actions -> 92 events, latest: {'Date': '2026-08-11T00:00:00.000', 'Dividends': 0.26,
'Stock Splits': 0.0}
```

What to notice

- `Client(url)` speaks the same protocol the agent will use. `list_tools()` is discovery; `call_tool(name, args)` is one JSON-RPC request. `.content[0].text` is our JSON string.
- A line like `server said: Error ...` means MCP is fine and the problem is Yahoo (rate limit, bad ticker, no network).

Step 3 — First agent: Claude + MCP tools

One file, three parts: configuration, a `log()` helper, and a `main()` that sends the prompt and reads the message stream.

`agent_plain.py`

```
"""Agent 1: Claude + yfinance MCP tools. No skills.

    python agent_plain.py "How has TCS.NS performed over the past year?"

Needs the MCP server running:  python yfinance_server.py
(http://127.0.0.1:8000/mcp)
Optional LangSmith tracing:    export LANGSMITH_TRACING=true LANGSMITH_API_KEY=...
LANGSMITH_PROJECT=yfinance-agent
"""

import asyncio
import os
import sys
from datetime import datetime

from claude_agent_sdk import (
    AssistantMessage,
    ClaudeAgentOptions,
    ClaudeSDKClient,
    ResultMessage,
    SystemMessage,
    ToolResultBlock,
    ToolUseBlock,
    UserMessage,
)

if os.environ.get("LANGSMITH_TRACING") == "true":
    from langsmith.integrations.claude_agent_sdk import configure_claude_agent_sdk
    configure_claude_agent_sdk(name="agent_plain")

# ---------------------------------------------------------------- configuration
MCP_URL = os.environ.get("YF_MCP_URL", "http://127.0.0.1:8000/mcp")

options = ClaudeAgentOptions(
    system_prompt=(
        "You are a market-data analyst. Use the yfinance tools for every number you
report; "
        "never recall prices or fundamentals from memory. If a tool returns an
error or no data, say so plainly."
    ),
    # the running FastMCP server; tools become mcp__yfinance__<tool_name>
    mcp_servers={"yfinance": {"type": "http", "url": MCP_URL}},
    # MCP tool calls need approval; this allow rule approves every tool of the
yfinance server
    allowed_tools=["mcp__yfinance__*"],
    # do not read .claude/ from disk -> no skills discovered. Without this line the
SDK
    # would find .claude/skills/ in the working directory and this agent would use
them.
    setting_sources=[],
    max_turns=20,
)
```

```python
# ---------------------------------------------------------------- run
prompt = " ".join(sys.argv[1:]) or "Give me a snapshot of RELIANCE.NS"
os.makedirs("logs", exist_ok=True)
log_file = open(f"logs/plain_{datetime.now():%Y%m%d_%H%M%S}.log", "w")


def log(event: str):
    line = f"{datetime.now():%H:%M:%S}  {event}"
    print(line)
    log_file.write(line + "\n")


async def main():
    log(f"PROMPT       {prompt}")
    async with ClaudeSDKClient(options=options) as client:
        await client.query(prompt)
        async for msg in client.receive_response():

            if isinstance(msg, SystemMessage) and msg.subtype == "init":
                log(f"SESSION      model={msg.data.get('model')}  mcp=
{msg.data.get('mcp_servers')}")
                log(f"SKILLS       {msg.data.get('skills') or 'none'}")

            elif isinstance(msg, AssistantMessage):
                for block in msg.content:
                    if isinstance(block, ToolUseBlock):
                        log(f"TOOL CALL    {block.name}  {block.input}")

            elif isinstance(msg, UserMessage) and isinstance(msg.content, list):
                for block in msg.content:
                    if isinstance(block, ToolResultBlock):
                        size = len(str(block.content))
                        log(f"TOOL RESULT  {'ERROR' if block.is_error else 'ok'}
{size} chars")

            elif isinstance(msg, ResultMessage):
                u = msg.usage or {}
                log(f"DONE         turns={msg.num_turns}  cost=${msg.total_cost_usd
or 0:.4f}  "
                    f"tokens: in={u.get('input_tokens', 0)} out=
{u.get('output_tokens', 0)} "
                    f"cache_read={u.get('cache_read_input_tokens', 0)}")
                answer = msg.result or ""

    print("\n" + "=" * 80 + "\n" + answer + "\n" + "=" * 80)
    log_file.write("\n" + answer + "\n")
    log_file.close()


asyncio.run(main())
```

`ClaudeAgentOptions` — the parameters used

Parameter	What it does here
`system_prompt`	Replaces the default Claude Code system prompt. Kept short so Step 6's comparison isn't confounded by prompt engineering.
`mcp_servers={"yfinance": {"type": "http", "url": ...}}`	Registers the running server. The key `yfinance` becomes the namespace: tools appear to the model as `mcp__yfinance__<tool>`. `type` can be `http`, `sse`, or `stdio` (with `command` / `args`).
`allowed_tools=["mcp__yfinance__*"]`	**Required.** MCP tool calls need approval; in a headless script there is nobody to approve, so an unlisted tool call is denied. The glob after the literal `mcp__yfinance__` prefix approves every tool of that server in one entry.
`setting_sources=[]`	Do not read `.claude/` from the working directory. Without this the SDK would discover `.claude/skills/` and this agent would stop being "MCP only".
`max_turns`	Cap on model turns; each tool round-trip is a turn.

The message stream — `client.receive_response()` yields typed messages. The agent logs one line per event:

Message	Log line	Meaning
`SystemMessage(subtype="init")`	`SESSION`, `SKILLS`	Session context: model, MCP server connection status, skills discovered (none here).
`AssistantMessage` → `ToolUseBlock`	`TOOL CALL`	The tool the model chose and the arguments it picked (watch `period`, `interval`).
`UserMessage` → `ToolResultBlock`	`TOOL RESULT`	What came back from the server — ok/error and size.
`ResultMessage`	`DONE`	Turn count, total cost, cumulative tokens; `.result` is the final answer.

`ClaudeSDKClient` is the SDK's session object: `query()` sends a prompt, `receive_response()` streams everything until the final `ResultMessage`. (The SDK also has a one-shot `query()` function; the client form is used here because the LangSmith integration in Step 7 hooks into it.)

Step 4 — Run the plain agent

Server running in Terminal 1. In Terminal 2:

macOS

```
uv run agent_plain.py "Give me a snapshot of RELIANCE.NS"
uv run agent_plain.py "How has TCS.NS performed over the past year?"
uv run agent_plain.py "Anything material in the news on AAPL? Why did it move this week?"
```

PowerShell

```
uv run agent_plain.py "Give me a snapshot of RELIANCE.NS"
uv run agent_plain.py "How has TCS.NS performed over the past year?"
uv run agent_plain.py "Anything material in the news on AAPL? Why did it move this week?"
```

Sample output for the second query:

```
21:40:12  PROMPT       How has TCS.NS performed over the past year?
21:40:14  SESSION      model=claude-sonnet-4-5  mcp=[{'name': 'yfinance', 'status': 'connected'}]
21:40:14  SKILLS       none
21:40:17  TOOL CALL    mcp__yfinance__get_historical_stock_prices  {'ticker': 'TCS.NS', 'period': '1y', 'interval': '1wk'}
21:40:19  TOOL RESULT  ok  9814 chars
21:40:31  DONE         turns=2  cost=$0.0312  tokens: in=8 out=510 cache_read=9130

================================================================================
TCS has had a difficult year. Starting from around INR 4,100 in November 2025 ...
================================================================================
```

Run the same query **three times** and compare `logs/plain_*.log`. Watch for:

- `period` / `interval` changing between runs (`1y/1d` vs `1y/1wk` vs `6mo`)
- Different metrics chosen — sometimes total return only, sometimes a price table, sometimes volatility
- Return figures computed in the model's head; rounding and even the sign of a drawdown can differ
- Different structure and length each time

This variance is the baseline for Step 6.

Step 5 — Second agent: Claude + MCP tools + Skills

Skills are folders under `<cwd>/.claude/skills/<name>/SKILL.md`. At start-up the SDK reads each skill's frontmatter (name + description) and lists them; the full body is loaded only when the model decides a skill applies. Three skills, each showing a different consistency mechanism.

5a. `stock-snapshot` — template-driven

`.claude/skills/stock-snapshot/SKILL.md`

````
---
name: stock-snapshot
description: One-page fundamental fact sheet for a single stock — price, valuation, profitability, balance sheet, dividends, analyst view. Use when asked for a snapshot, overview, fact sheet, fundamentals, "tell me about <ticker>", or "what does <company> look like".
---

# Stock Snapshot

## Procedure
1. Call `mcp__yfinance__get_stock_info` with the ticker.
2. Call `mcp__yfinance__get_stock_actions` with the ticker.
3. Fill the template below. Do not add sections; do not drop sections. Use exactly the fields listed.

## Formatting rules
- Missing/null field → write `n/a`. Never estimate or fill from memory.
- Prices and targets: `{currency} {value:,.2f}` (e.g. `INR 2,890.50`, `USD 232.40`).
- Market cap: in billions with one decimal and currency, e.g. `INR 19,560.0 bn`.
- Ratios (PE, P/B, EV/EBITDA, current ratio): one decimal, e.g. `27.8x`.
- Percentages (margins, ROE, growth, yield, payout): one decimal with `%`. Yahoo gives these as fractions — multiply by 100. `debtToEquity` is already a percentage; show as-is with one decimal and `%`.
- Dividends: last 3 entries from actions with `Dividends > 0`, newest first, `YYYY-MM-DD: {currency} {amount}`. Splits: all entries with `Stock Splits > 0`, newest first, `YYYY-MM-DD: {ratio}:1`. If none, write `none on record`.
- No commentary, no recommendation of your own, no closing remarks. The template is the entire answer.

## Template (copy exactly)

```
## {longName} ({symbol})
{sector} · {industry}

**Price**
- Last: {currency} {currentPrice} (prev close {currency} {previousClose})
- 52-week range: {currency} {fiftyTwoWeekLow} – {fiftyTwoWeekHigh}
- Market cap: {marketCap}

**Valuation**
- Trailing P/E: {trailingPE} · Forward P/E: {forwardPE}
- Price/Book: {priceToBook} · EV/EBITDA: {enterpriseToEbitda}

**Profitability & growth**
- Operating margin: {operatingMargins} · Net margin: {profitMargins}
- ROE: {returnOnEquity}
- Revenue growth (yoy): {revenueGrowth} · Earnings growth (yoy): {earningsGrowth}

**Balance sheet**
````
````

```
- Debt/Equity: {debtToEquity} · Current ratio: {currentRatio}

**Shareholder returns**
- Dividend yield: {dividendYield} · Payout ratio: {payoutRatio}
- Recent dividends: {last 3}
- Splits: {list}

**Analyst view**
- Consensus: {recommendationKey} ({numberOfAnalystOpinions} analysts) · Mean
target: {currency} {targetMeanPrice}

_Source: Yahoo Finance via MCP. Data as retrieved; not investment advice._
```

5b. `price-performance` — script-driven

The skill forbids mental arithmetic: a bundled script fetches prices from the **same MCP server** (it is an MCP client too) and computes the metrics. The model only picks the period and fills the template with the script's numbers.

`.claude/skills/price-performance/SKILL.md`

````
---
name: price-performance
description: Price performance and trend of one or more stocks — total return, CAGR, volatility, max drawdown, moving-average trend. Use when asked how a stock has done / performed, whether it is in an uptrend or downtrend, how volatile it is, or to compare returns of several tickers.
---

# Price Performance

## Procedure
1. Pick the period from the question: "this year" → `ytd`; "past year"/unspecified → `1y`; "6 months" → `6mo`; "3 months" → `3mo`; "5 years" → `5y`.
2. Run the metrics script (it fetches daily prices from the yfinance MCP server and computes everything):
   ```
   python .claude/skills/price-performance/scripts/metrics.py TICKER [TICKER ...] --period PERIOD
   ```
   It prints one JSON object per ticker. Do NOT call `get_historical_stock_prices` yourself and do NOT compute returns by hand — use the script's numbers verbatim.
3. Fill the template. For several tickers, one block per ticker in the order asked, then the comparison table.

## Formatting rules
- Returns, volatility, drawdown: one decimal with `%` and sign (`+14.2%`, `-8.7%`).
- Prices: `{currency} {value:,.2f}`.
- Trend label comes from the script (`uptrend` / `downtrend` / `sideways`) — do not override it.
- If the script reports an error for a ticker, output `## {TICKER}` followed by the error line and nothing else for that ticker.
- No forecasts, no buy/sell language, no commentary beyond the template.

## Template (per ticker)

```
## {TICKER} — {period} performance
- Window: {start_date} → {end_date} ({n_days} trading days)
- Price: {start_close} → {end_close}
- Total return: {total_return} · CAGR: {cagr}
- Annualised volatility: {volatility}
- Max drawdown: {max_drawdown} ({dd_start} → {dd_trough})
- Best / worst month: {best_month} ({best_month_ret}) / {worst_month} ({worst_month_ret})
- Trend: **{trend}** — price is {pct_vs_50dma} vs 50-day avg and {pct_vs_200dma} vs 200-day avg
```

## Comparison table (only when 2+ tickers)

```
| Ticker | Total return | CAGR | Volatility | Max drawdown | Trend |
````
````

````
|---|---|---|---|---|---|
```

End every answer with:
`_Source: Yahoo Finance via MCP; metrics computed by price-performance/metrics.py. Not investment advice._`
````

`.claude/skills/price-performance/scripts/metrics.py`
````

```
"""Fetch daily prices from the yfinance MCP server and compute performance metrics.

python metrics.py RELIANCE.NS TCS.NS --period 1y
Server URL from $YF_MCP_URL (default http://127.0.0.1:8000/mcp).
"""

import argparse
import asyncio
import json
import math
import os

from fastmcp import Client

MCP_URL = os.environ.get("YF_MCP_URL", "http://127.0.0.1:8000/mcp")


def pct(x):
    return f"{x * 100:+.1f}%"


def compute(rows, period):
    closes = [r["Close"] for r in rows]
    dates = [r["Date"][:10] for r in rows]
    n = len(closes)
    start, end = closes[0], closes[-1]
    total = end / start - 1
    years = max((n - 1) / 252, 1 / 252)
    cagr = (end / start) ** (1 / years) - 1 if years >= 0.5 else None
    rets = [closes[i] / closes[i - 1] - 1 for i in range(1, n)]
    mean = sum(rets) / len(rets)
    vol = math.sqrt(sum((r - mean) ** 2 for r in rets) / max(len(rets) - 1, 1)) *
math.sqrt(252)

    peak, peak_i, mdd, dd_start, dd_trough = closes[0], 0, 0.0, dates[0], dates[0]
    for i, c in enumerate(closes):
        if c > peak:
            peak, peak_i = c, i
        dd = c / peak - 1
        if dd < mdd:
            mdd, dd_start, dd_trough = dd, dates[peak_i], dates[i]

    months = {}
    for d, c in zip(dates, closes):
        months.setdefault(d[:7], []).append(c)
    mrets = {m: v[-1] / v[0] - 1 for m, v in months.items() if len(v) > 1}
    best = max(mrets, key=mrets.get) if mrets else None
    worst = min(mrets, key=mrets.get) if mrets else None

    def sma(k):
        return sum(closes[-k:]) / k if n >= k else None
```

```
    s50, s200 = sma(50), sma(200)
    v50 = end / s50 - 1 if s50 else None
    v200 = end / s200 - 1 if s200 else None
    if v50 is None:
        trend = "sideways"
    elif v50 > 0.02 and (v200 is None or v200 > 0):
        trend = "uptrend"
    elif v50 < -0.02 and (v200 is None or v200 < 0):
        trend = "downtrend"
    else:
        trend = "sideways"

    return {
        "period": period, "start_date": dates[0], "end_date": dates[-1], "n_days":
n,
        "start_close": round(start, 2), "end_close": round(end, 2),
        "total_return": pct(total), "cagr": pct(cagr) if cagr is not None else "n/a
(window < 6 months)",
        "volatility": pct(vol), "max_drawdown": pct(mdd), "dd_start": dd_start,
"dd_trough": dd_trough,
        "best_month": best or "n/a", "best_month_ret": pct(mrets[best]) if best
else "n/a",
        "worst_month": worst or "n/a", "worst_month_ret": pct(mrets[worst]) if
worst else "n/a",
        "trend": trend,
        "pct_vs_50dma": pct(v50) if v50 is not None else "n/a", "pct_vs_200dma":
pct(v200) if v200 is not None else "n/a",
    }


async def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("tickers", nargs="+")
    ap.add_argument("--period", default="1y")
    a = ap.parse_args()
    out = {}
    async with Client(MCP_URL) as c:
        for t in a.tickers:
            r = await c.call_tool("get_historical_stock_prices", {"ticker": t,
"period": a.period, "interval": "1d"})
            raw = r.content[0].text
            try:
                rows = json.loads(raw)
            except json.JSONDecodeError:
                out[t] = {"error": raw}
                continue
            out[t] = compute(rows, a.period) if len(rows) >= 2 else {"error":
"fewer than 2 rows"}
    print(json.dumps(out, indent=1))
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Test the script by itself before involving the agent:

macOS

```
uv run .claude/skills/price-performance/scripts/metrics.py TCS.NS --period 1y
```

PowerShell

```
uv run .claude\skills\price-performance\scripts\metrics.py TCS.NS --period 1y
```

5c. `news-brief` — procedure-driven

`.claude/skills/news-brief/SKILL.md`

````
---
name: news-brief
description: Triage and summarise recent news for a stock by materiality, and
optionally link headlines to recent price moves. Use when asked what's in the news,
anything material, recent developments, a news summary, or "why did <ticker> move /
fall / rise".
---

# News Brief

## Procedure
1. Call `mcp__yfinance__get_yahoo_finance_news` with the ticker.
2. If the question asks *why* the price moved, also call
`mcp__yfinance__get_historical_stock_prices` with `period="5d", interval="1d"` and
compute each day's close-to-close % change.
3. Classify every story into exactly one category:
   `EARNINGS` · `GUIDANCE` · `M&A` · `REGULATORY` · `MANAGEMENT` · `PRODUCT` ·
`MACRO/SECTOR` · `NOISE`
   `NOISE` = listicles, "stocks to watch", generic market wraps, opinion pieces
with no new facts.
4. Materiality: `HIGH` = EARNINGS, GUIDANCE, M&A, REGULATORY with a stated
financial impact; `MEDIUM` = other company-specific facts; `LOW` = MACRO/SECTOR;
NOISE is dropped.
5. Fill the template. Keep each summary to one sentence, in your own words, from
the article's title/summary only — do not add facts you did not retrieve.

## Template

```
## {TICKER} — news brief ({count_kept} of {count_total} stories material)

**High**
- [{CATEGORY}] {YYYY-MM-DD} — {one-sentence summary}

**Medium**
- [{CATEGORY}] {YYYY-MM-DD} — {one-sentence summary}

**Low**
- [{CATEGORY}] {YYYY-MM-DD} — {one-sentence summary}

**Dropped as noise:** {count_noise}
```

If a tier has no items, write `- none` under it.

## Only when asked "why did it move"
Append:

```
**Price context (last 5 sessions)**
| Date | Close | Change |
|---|---|---|
````
````

```
**Likely drivers:** {1–2 sentences linking the largest move(s) to the HIGH/MEDIUM
items dated on or just before it; if none line up, say "no company-specific
catalyst in the retrieved news".}
```

End every answer with:
`_Source: Yahoo Finance via MCP. Headlines as retrieved; not investment advice._`

5d. The agent

`agent_skills.py` — same shape as `agent_plain.py`; the differences are marked in the configuration block and the log loop distinguishes `SKILL LOAD` and `BASH` events.

```
"""Agent 2: Claude + yfinance MCP tools + skills from .claude/skills/.

    python agent_skills.py "How has TCS.NS performed over the past year?"

Needs the MCP server running:  python yfinance_server.py
(http://127.0.0.1:8000/mcp)
Optional LangSmith tracing:    export LANGSMITH_TRACING=true LANGSMITH_API_KEY=...
LANGSMITH_PROJECT=yfinance-agent
"""

import asyncio
import os
import sys
from datetime import datetime

from claude_agent_sdk import (
    AssistantMessage,
    ClaudeAgentOptions,
    ClaudeSDKClient,
    ResultMessage,
    SystemMessage,
    ToolResultBlock,
    ToolUseBlock,
    UserMessage,
)

if os.environ.get("LANGSMITH_TRACING") == "true":
    from langsmith.integrations.claude_agent_sdk import configure_claude_agent_sdk
    configure_claude_agent_sdk(name="agent_skills")

# ---------------------------------------------------------------- configuration
MCP_URL = os.environ.get("YF_MCP_URL", "http://127.0.0.1:8000/mcp")
PROJECT_DIR = os.environ.get("PROJECT_DIR", os.getcwd())   # folder that contains
.claude/skills/

options = ClaudeAgentOptions(
    system_prompt=(
        "You are a market-data analyst. Use the yfinance tools for every number you
report; "
        "never recall prices or fundamentals from memory. If a tool returns an
error or no data, say so plainly."
    ),
    mcp_servers={"yfinance": {"type": "http", "url": MCP_URL}},
    allowed_tools=[
        "mcp__yfinance__*",   # every tool of the yfinance server
        "Bash",               # lets a skill run its script (metrics.py)
    ],
    cwd=PROJECT_DIR,                  # skills are discovered at
<cwd>/.claude/skills/
    setting_sources=["project"],      # read .claude/ from the project directory
    skills="all",                     # every discovered skill may be invoked
    env={"YF_MCP_URL": MCP_URL},      # so metrics.py (run via Bash) finds the same
```

```
server
    max_turns=20,
)

# ---------------------------------------------------------------- run
prompt = " ".join(sys.argv[1:]) or "Give me a snapshot of RELIANCE.NS"
os.makedirs("logs", exist_ok=True)
log_file = open(f"logs/skills_{datetime.now():%Y%m%d_%H%M%S}.log", "w")


def log(event: str):
    line = f"{datetime.now():%H:%M:%S}  {event}"
    print(line)
    log_file.write(line + "\n")


async def main():
    log(f"PROMPT       {prompt}")
    async with ClaudeSDKClient(options=options) as client:
        await client.query(prompt)
        async for msg in client.receive_response():

            if isinstance(msg, SystemMessage) and msg.subtype == "init":
                log(f"SESSION      model={msg.data.get('model')}  mcp=
{msg.data.get('mcp_servers')}")
                log(f"SKILLS       {msg.data.get('skills') or 'none'}")

            elif isinstance(msg, AssistantMessage):
                for block in msg.content:
                    if isinstance(block, ToolUseBlock):
                        if block.name == "Skill":
                            log(f"SKILL LOAD   {block.input.get('skill')}")
                        elif block.name == "Bash":
                            log(f"BASH         {block.input.get('command')}")
                        else:
                            log(f"TOOL CALL    {block.name}  {block.input}")

            elif isinstance(msg, UserMessage) and isinstance(msg.content, list):
                for block in msg.content:
                    if isinstance(block, ToolResultBlock):
                        size = len(str(block.content))
                        log(f"TOOL RESULT  {'ERROR' if block.is_error else 'ok'}
{size} chars")

            elif isinstance(msg, ResultMessage):
                u = msg.usage or {}
                log(f"DONE         turns={msg.num_turns}  cost=${msg.total_cost_usd
or 0:.4f}  "
                    f"tokens: in={u.get('input_tokens', 0)} out=
{u.get('output_tokens', 0)} "
                    f"cache_read={u.get('cache_read_input_tokens', 0)}")
                answer = msg.result or ""
```

```
    print("\n" + "=" * 80 + "\n" + answer + "\n" + "=" * 80)
    log_file.write("\n" + answer + "\n")
    log_file.close()


asyncio.run(main())
```

What changed in `ClaudeAgentOptions`

Parameter	Plain	Skills	Why
`cwd`	—	`PROJECT_DIR`	Skills are discovered at `<cwd>/.claude/skills/`.
`setting_sources`	`[]`	`["project"]`	`project` = read `<cwd>/.claude/`. (`"user"` would add `~/.claude/skills/`.)
`skills`	—	`"all"`	Which discovered skills the model may invoke. A list of names restricts it. Setting this also adds the `Skill` tool to the allow list, so it need not be listed.
`allowed_tools`	`mcp__yfinance__*`	+ `Bash`	`Bash` runs `metrics.py`. (`Read` is not needed: file reads inside `cwd` never require approval.)
`env`	—	`YF_MCP_URL`	Reaches `metrics.py` through Bash so the script uses the same server.

Environment variables

Variable	Default	Purpose
`PROJECT_DIR`	current working directory	Folder that contains `.claude/skills/`. Set it when running the agent from elsewhere.
`YF_MCP_URL`	`http://127.0.0.1:8000/mcp`	Where the MCP server is. Used by both agents, `test_client.py` and `metrics.py`.

macOS

```
export PROJECT_DIR=/path/to/yfinance-agent
export YF_MCP_URL=http://127.0.0.1:8000/mcp        # change host/port/https here if needed
```

PowerShell

```
$env:PROJECT_DIR = "C:\path\to\yfinance-agent"
$env:YF_MCP_URL  = "http://127.0.0.1:8000/mcp"
```

Running from inside the project folder (`cd yfinance-agent`) needs neither — the defaults cover it.

SKILL.md anatomy

- Frontmatter `name` must match the directory name. `description` is the *only* thing the model sees before deciding to invoke — put trigger phrases there.
- The body is loaded on invocation and becomes instructions in context. Three levers: a fixed **procedure** (which tools, in what order, with what arguments), **formatting rules** (units, decimals, missing values), and a literal **template**.
- `scripts/` is ordinary code the model runs with Bash. Anything numeric or deterministic belongs there, not in the prompt.

Step 6 — Run the skills agent with the same queries

macOS

```
uv run agent_skills.py "Give me a snapshot of RELIANCE.NS"
uv run agent_skills.py "How has TCS.NS performed over the past year?"
uv run agent_skills.py "Anything material in the news on AAPL? Why did it move this
week?"
```

PowerShell

```
uv run agent_skills.py "Give me a snapshot of RELIANCE.NS"
uv run agent_skills.py "How has TCS.NS performed over the past year?"
uv run agent_skills.py "Anything material in the news on AAPL? Why did it move this
week?"
```

Sample output for the second query:

```
21:52:03  PROMPT       How has TCS.NS performed over the past year?
21:52:05  SESSION      model=claude-sonnet-4-5  mcp=[{'name': 'yfinance', 'status': 'connected'}]
21:52:05  SKILLS       ['news-brief', 'price-performance', 'stock-snapshot']
21:52:08  SKILL LOAD   price-performance
21:52:08  TOOL RESULT  ok  2912 chars
21:52:12  BASH         python .claude/skills/price-performance/scripts/metrics.py TCS.NS --period
1y
21:52:16  TOOL RESULT  ok  687 chars
21:52:24  DONE         turns=3  cost=$0.0418  tokens: in=12 out=640 cache_read=14210

================================================================================
## TCS.NS — 1y performance
- Window: 2025-10-29 → 2026-10-28 (248 trading days)
- Price: INR 4,105.20 → INR 3,412.00
- Total return: -16.9% · CAGR: -17.0%
- Annualised volatility: +22.1%
- Max drawdown: -31.2% (2025-12-04 → 2026-08-19)
- Best / worst month: 2026-02 (+7.5%) / 2025-12 (-11.2%)
- Trend: **downtrend** — price is -3.0% vs 50-day avg and -12.0% vs 200-day avg

_Source: Yahoo Finance via MCP; metrics computed by price-performance/metrics.py. Not investment
advice._
================================================================================
```

Run each query three times and compare `logs/skills_*.log` with `logs/plain_*.log`:

	Plain agent	Skills agent
Event sequence	`TOOL CALL` with arguments the model picked	`SKILL LOAD` → fixed procedure (`BASH metrics.py` for performance)
Numbers	Computed by the model from raw JSON	Computed by code; identical across runs on identical data
Structure	Different headings/length each run	Same template every run
Tokens	Smaller	~2–3k more per invoked skill (the SKILL.md body)
`SKILLS` line	`none`	the three skill names

The event log is the clearest explanation of what a skill changes: the *procedure* moves out of the model's discretion and into a file.

Step 7 — Optional: trace both agents in LangSmith

LangSmith has a native integration for the Claude Agent SDK. Both agents already contain the hook; it activates only when `LANGSMITH_TRACING=true`.

macOS

```
uv add "langsmith[claude-agent-sdk]"
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=lsv2_...
export LANGSMITH_PROJECT=yfinance-agent
uv run agent_skills.py "How has TCS.NS performed over the past year?"
```

PowerShell

```
uv add "langsmith[claude-agent-sdk]"
$env:LANGSMITH_TRACING = "true"
$env:LANGSMITH_API_KEY = "lsv2_..."
$env:LANGSMITH_PROJECT = "yfinance-agent"
uv run agent_skills.py "How has TCS.NS performed over the past year?"
```

The relevant lines in each agent:

```
if os.environ.get("LANGSMITH_TRACING") == "true":
    from langsmith.integrations.claude_agent_sdk import configure_claude_agent_sdk
    configure_claude_agent_sdk(name="agent_skills")
```

`configure_claude_agent_sdk()` patches `ClaudeSDKClient` in place and installs `PreToolUse` / `PostToolUse` hooks, so each run appears in LangSmith as a chain run containing one model run per assistant turn and one tool run per call — MCP tools, `Skill` and `Bash` included — with inputs, outputs and token counts. Open the project in LangSmith and compare a plain run with a skills run of the same query: the extra `Skill` and `Bash` spans are the skill at work.

Troubleshooting

Symptom	Cause / fix
`SESSION ... mcp=[{'name': 'yfinance', 'status': 'failed'}]`	Server not running on port 8000, or `YF_MCP_URL` wrong. Re-run Step 2's client.
`SKILLS none` in the skills agent	`PROJECT_DIR` (or the current directory) doesn't contain `.claude/skills/`, or `setting_sources` lacks `"project"`.
Model answers without calling any tool, or a tool call is denied	`allowed_tools` must contain `mcp__yfinance__*` (or the full `mcp__yfinance__<tool>` names).
`server said: Error ... Could not resolve host`	Machine can't reach Yahoo. MCP is fine; network/rate-limit issue.
Skills agent stalls	A tool not in `allowed_tools` was requested (e.g. `Write`). Add it or tighten the skill.
HTTPS with self-signed cert fails	`NODE_EXTRA_CA_CERTS=cert.pem` (Claude Code process) and `SSL_CERT_FILE=cert.pem` (for `metrics.py`).
No traces in LangSmith	`LANGSMITH_TRACING` must be exactly `true`; check `LANGSMITH_API_KEY`; the agent must use `ClaudeSDKClient`, not the one-shot `query()`.

Extensions

- Add a fourth skill `peer-compare` that composes `stock-snapshot` fields across 2–4 tickers into one table.
- Set `skills=["price-performance"]` and ask a snapshot question — observe the fallback.
- Paste a SKILL.md body into the plain agent's `system_prompt` and compare: what does the skills mechanism buy beyond a long prompt? (Loaded only when relevant, modular, ships scripts.)